

Engenharia de Detecção e Análise Forense: Implementação Wazuh e TShark

Este repositório consolida a minha experiência técnica na configuração de sistemas de detecção (SIEM/HIDS) e análise de tráfego de rede, priorizando a visibilidade sobre táticas adversárias e a otimização da fidelidade dos alertas.

1. Análise Estatística de Rede (TShark)

Objetivo: Identificar anomalias de tráfego e infraestrutura sem interface gráfica.

Utilizei o **TShark** para extrair metadados de pacotes, focando na identificação de vetores de ataque através de estatísticas de protocolo:

- **Hierarquia de Protocolos (phs):** Identificação de tráfego anômalo (ex: túneis ICMP ou tráfego HTTP em portas não padrão).
 - **Análise de Endpoints e Conversas:** Mapeamento de "Top Talkers" para detetar exfiltração de dados ou varreduras de rede (Network Scanning).
 - **Expert Info:** Extração de diagnósticos automáticos para identificar retransmissões TCP, indicativas de possíveis ataques de DoS ou problemas de conectividade.
-

2. Estruturação de Logs: Decodificadores Sysmon

Objetivo: Normalizar telemetria Windows para análise granular.

Implementei decodificadores para transformar logs brutos do **Microsoft Sysmon** em campos estruturados. Este processo é a fundação da detecção, permitindo que o Wazuh identifique:

- `sysmon.image`: O executável em execução.
 - `sysmon.parentImage`: O processo pai, essencial para detetar **Process Injection**.
 - `sysmon.commandLine`: Os argumentos utilizados (ex: scripts PowerShell codificados em Base64).
-

3. Lógica de Correlação e Hierarquia de Regras

Objetivo: Implementar árvores de decisão para reduzir falsos positivos.

Utilizei a funcionalidade `if_sid` para criar dependências lógicas entre alertas. Esta abordagem permite que o SIEM ignore atividades administrativas legítimas e foque em desvios comportamentais.

- **Deteção Crítica:** Execução de `svchost.exe` com origem no utilizador (`explorer.exe`).
 - **Tuning (Supressão):** Atribuição de **Nível 0** para `svchost.exe` iniciado pelo `services.exe`, eliminando o ruído operacional e focando a equipa de resposta em ameaças reais.
-

4. Custom Rules: Monitorização de Integridade em Linux

Objetivo: Deteção de ferramentas de pós-exploração via Auditd.

Desenvolvi regras locais (`local_rules.xml`) para monitorizar syscalls do kernel Linux, focando na tática de **Persistence** e **Defense Evasion**:

- **Monitorização de Diretórios Temporários:** Alerta automático para a criação de ficheiros em `/tmp`, `/var/tmp` e `downloads`.
 - **Análise de Extensões:** Regras específicas para scripts (`.sh`, `.py`) e binários (`.elf`).
 - **Deteção de Assinaturas:** Identificação de nomes de ferramentas conhecidas (ex: `linpeas`, `mimipenguin`).
-

5. Engenharia de Deteção: O Funil de Alertas

Objetivo: Automatizar a triagem de incidentes.

A estrutura de regras foi desenhada como um funil:

1. **Visibilidade:** Regras base de baixa severidade para registo histórico.
 2. **Análise:** Regras filhas que validam extensões e permissões.
 3. **Ação:** Regras críticas (Nível 12+) mapeadas para o **MITRE ATT&CK** (ex: T1105 - Ingress Tool Transfer).
 4. **Exceções:** Regras de Nível 0 para ferramentas de auditoria e Red Team previamente aprovadas.
-

Conclusão

A documentação aqui apresentada reflete uma metodologia de trabalho baseada em evidências. Ao unir a profundidade analítica do TShark com a capacidade de escala do Wazuh, demonstro proficiência na criação de um ambiente de monitorização resiliente, capaz de distinguir operações legítimas de intrusões sofisticadas.

Relatório Técnico: Extração de Estatísticas Avançadas por Protocolo via TShark

1. Contexto e Cenário

A análise de tráfego de rede em larga escala exige ferramentas que permitam a síntese de dados sem a necessidade de processamento gráfico intensivo. Este relatório detalha a utilização do TShark como motor de análise estatística para protocolos específicos (IPv4/v6, DNS e HTTP), focando na identificação de padrões de comunicação e anomalias em infraestruturas críticas.

2. Objetivo Técnico

Demonstrar a proficiência na aplicação de filtros de estatística (`-z`) para gerar visibilidade sobre a camada de rede e aplicação, permitindo a correlação de hosts, análise de códigos de resposta DNS e monitorização de integridade de serviços web.

3. Ambiente Utilizado

- Analista de Redes/SOC: Ambiente CLI (Command Line Interface).
 - Ferramenta: TShark (v3.x ou superior).
 - Protocolos Analisados: IPv4, IPv6, DNS, HTTP/HTTP2.
-

4. Implementação de Filtros Estatísticos

4.1. Camada de Rede (IPv4 e IPv6)

A análise de endpoints e a distribuição de protocolos de transporte são fundamentais para o mapeamento da superfície de ataque e fluxos de dados internos.

- Distribuição de Protocolos: Através do comando `-z ptype, tree`, é possível identificar a volumetria de TCP vs. UDP, auxiliando na deteção de tráfego não esperado (ex: excesso de UDP em servidores de banco de dados).
- Mapeamento de Hosts: O filtro `-z ip_hosts, tree` consolida todos os endereços IPs ativos na captura, facilitando a rápida identificação de "Top Talkers".
- Direcionalidade (Origem/Destino): A utilização de `-z ip_srcdst, tree` permite correlacionar fluxos de exfiltração de dados (Inbound vs. Outbound).

4.2. Análise de Resolução de Nomes (DNS)

A monitorização do protocolo DNS é crítica para identificar técnicas de C2 (Command & Control) e exfiltração por túneis DNS.

- Integridade de Respostas: O parâmetro `-z dns, tree` resume os códigos de erro (rcodes) e códigos de operação (opcodes).
- Aplicação em SOC: Um volume anormal de respostas *NXDOMAIN* ou *Standard Query* para domínios de alta entropia pode indicar a presença de malwares utilizando algoritmos DGA (Domain Generation Algorithm).

4.3. Monitorização de Tráfego de Aplicação (HTTP/HTTP2)

Para serviços web, o TShark oferece métricas detalhadas sobre a experiência do utilizador e a saúde do servidor.

- Status de Resposta: O filtro `-z http, tree` categoriza respostas em sucessos (2xx), redirecionamentos (3xx) e erros (4xx/5xx).
 - Distribuição de Carga: Com `-z http_srv, tree`, identificamos quais recursos ou servidores estão sob maior pressão de requisições.
-

5. Justificativa das Decisões Técnicas

- Uso de Árvores Estatísticas (`tree`): Optou-se por este formato para fornecer uma visão hierárquica e percentual, permitindo que o analista tome decisões baseadas na relevância estatística do evento.
- Parâmetro `-q` (Quiet): Essencial para suprimir a impressão de pacotes individuais e focar exclusivamente no output do relatório, otimizando a performance em capturas volumosas.

6. Impacto Operacional para SOC

A transição da análise manual de pacotes para a síntese estatística via CLI resulta em:

- Redução do MTDD (Mean Time to Detect): Identificação de IPs maliciosos ou serviços instáveis em segundos.
- Eficiência em Investigação Forense: Capacidade de processar gigabytes de PCAPs em servidores remotos sem necessidade de exportar arquivos para máquinas locais.

7. Conclusão Técnica

A maestria sobre os sub-parâmetros do comando `-z` no TShark permite uma análise de segurança orientada a dados. A capacidade de decompor protocolos específicos em árvores de estatísticas é o que diferencia uma execução mecânica de uma investigação forense madura e competitiva.

Relatório de Engenharia de Detecção: Análise Avançada de Protocolos com TShark

1. Contexto e Cenário

Esta documentação detalha a utilização avançada do **TShark** para a extração de métricas específicas de protocolos (IPv4/IPv6, DNS e HTTP). O cenário pressupõe uma investigação de rede onde a identificação de volumes anómalos de tráfego ou códigos de status de erro é vital para a triagem de incidentes de segurança, como exfiltração de dados ou ataques de negação de serviço.

2. Objetivo Técnico

Demonstrar a proficiência na utilização do parâmetro `-z` para gerar árvores estatísticas detalhadas, permitindo ao analista de SOC (Security Operations Center) isolar hosts ofensivos, analisar a integridade de serviços web e auditar a resolução de nomes na infraestrutura.

3. Ambiente Utilizado

- **Ferramenta de Análise:** TShark (Terminal-based Wireshark)
- **Formatos de Arquivo:** PCAP, PCAPNG
- **Sistemas:** Linux (Ubuntu/Debian)

4. Arquitetura de Análise de Protocolos

A análise segue o modelo de inspeção profunda de pacotes (DPI) na linha de comando:

1. **Ingestão de Dados:** Leitura do ficheiro via `-r`.
2. **Filtragem de Estatísticas:** Aplicação de sub-filtros de protocolo via `-z [protocolo],tree`.
3. **Supressão de Ruído:** Utilização do parâmetro `-q` para evitar a exibição do fluxo individual de pacotes e focar exclusivamente no relatório consolidado.

5. Implementação Técnica de Filtros Específicos

5.1. Estatísticas de Camada de Rede (IPv4/IPv6)

A análise de endereçamento permite identificar a distribuição de protocolos de transporte e os "Top Talkers" da rede.

- **Tipos de Protocolo:** `-z ptype,tree` revela a proporção entre TCP, UDP e ICMP.
- **Mapeamento de Hosts:** `-z ip_hosts,tree` consolida todos os endereços IP presentes na captura.
- **Correlação Origem/Destino:** `-z ip_srcdst,tree` permite visualizar a direção do tráfego, essencial para detetar fluxos de exfiltração de dados (Outbound Data Flow).

5.2. Auditoria de Resolução de Nomes (DNS)

Através do comando `-z dns,tree`, extraímos um resumo operacional do serviço DNS.

- **Rcodes:** Permite identificar falhas de resolução (*NXDOMAIN*), que podem indicar tentativas de algoritmo de geração de domínio (DGA) por malwares.
- **Opcodes:** Garante que apenas queries padrão estão a ser executadas, detetando anomalias no protocolo.

5.3. Monitorização de Tráfego Web (HTTP/HTTP2)

A análise de serviços HTTP é crítica para entender a integridade de aplicações web expostas.

- **Códigos de Status:** `-z http,tree` resume sucessos (2xx), redirecionamentos (3xx) e erros (4xx/5xx).
 - **Distribuição de Carga:** `-z http_srv,tree` identifica quais servidores ou caminhos estão a receber maior volume de pedidos.
-

6. Justificativa das Decisões Técnicas

- **Uso de Árvores de Estatísticas:** Optou-se pela visualização em árvore (`tree`) para fornecer uma perspetiva hierárquica, facilitando a identificação imediata de percentagens de tráfego desproporcionais.
- **Filtros de Destino e Portas:** A utilização de `-z dests,tree` é estratégica para identificar serviços não autorizados ou o uso de portas não padrão para comunicações C2 (Command and Control).

7. Impacto Operacional para SOC

- **Velocidade de Triagem:** A capacidade de resumir milhares de pacotes DNS ou HTTP em segundos reduz drasticamente o tempo de investigação inicial.
- **Deteção de Anomalias:** O resumo de endereços IP de origem e destino facilita a criação rápida de listas de bloqueio (Blacklists) em Firewalls após a identificação de um host malicioso.

8. Considerações de Produção

- **Consumo de Memória:** Ao processar ficheiros PCAP muito extensos (>1GB), o TShark pode consumir recursos significativos para manter a tabela de estados. Em produção, recomenda-se segmentar a captura antes do processamento estatístico.
- **Filtros de Exibição:** Pode-se combinar as estatísticas com filtros de exibição padrão para isolar janelas temporais ou sub-redes específicas.

9. Conclusão Técnica

O domínio dos recursos de estatística do TShark eleva a capacidade do analista de SOC de "ver a floresta antes das árvores". A utilização estratégica de filtros de protocolo permite uma análise baseada em dados reais, sustentando decisões críticas de bloqueio e mitigação de ameaças.

Relatório Técnico: Operadores Avançados e Extração de Campos com TShark

1. Contexto e Cenário

Em investigações de segurança de alto nível, a filtragem por cabeçalhos estáticos é insuficiente. Este relatório detalha o uso de operadores lógicos avançados e a extração seletiva de campos no **TShark**, permitindo a busca de padrões específicos (Payload Inspection) e a estruturação de dados para análise forense simplificada.

2. Objetivo Técnico

Demonstrar a aplicação dos operadores `contains` e `matches` para inspeção de conteúdo e a utilização do modo `-T fields` para customizar a saída do terminal, facilitando a correlação de indicadores de comprometimento (IoCs).

3. Ambiente Utilizado

- **Ferramenta:** TShark (CLI)
 - **Protocolos Alvo:** IP, HTTP
 - **Sistemas:** Linux (Ubuntu/Debian)
-

4. Operadores de Comparação Avançada

4.1. Operador `contains` (Busca de Valor)

O operador `contains` é utilizado para localizar strings ou sequências de bytes específicas dentro de um campo de protocolo. É uma ferramenta fundamental para identificar assinaturas de servidores ou nomes de arquivos em trânsito.

- **Caraterística:** Case-sensitive (diferencia maiúsculas de minúsculas).
- **Aplicação:** Filtrar tráfego de servidores específicos (ex: Apache, Nginx) ou extensões de arquivos em requisições HTTP.

4.2. Operador `matches` (Expressões Regulares)

O operador `matches` introduz o suporte a PCRE (Perl Compatible Regular Expressions), permitindo buscas baseadas em padrões complexos.

- **Caraterística:** Case-insensitive (por padrão) e extremamente flexível.
 - **Aplicação:** Detetar múltiplas variações de métodos HTTP (GET/POST) ou padrões de exfiltração de dados que sigam uma lógica específica (ex: números de cartões de crédito ou tokens de autenticação).
-

5. Extração Seletiva de Campos (-T fields)

A extração de campos transforma o TShark de um "sniffer" num processador de dados estruturados. Ao utilizar `-T fields`, o analista define exatamente quais colunas deseja visualizar, eliminando o ruído das informações padrão da frame.

Estrutura da Implementação:

1. **-T fields:** Altera o modo de saída para campos específicos.
 2. **-e [campo]:** Especifica o campo a extrair (pode ser repetido para múltiplos campos).
 3. **-E header=y:** Adiciona uma linha de cabeçalho à saída, essencial para exportação para CSV/Excel.
-

6. Justificativa das Decisões Técnicas

- **Inibição de Inteiros:** Os operadores `contains` e `matches` não são aplicáveis a campos de valor inteiro (como `tcp.port`), sendo restritos a strings ou sequências de bytes. Esta distinção é crítica para evitar erros de sintaxe durante a investigação.
- **Uso de Hexadecimal:** Para aumentar a precisão da busca em payloads binários, recomenda-se o uso de valores HEX em vez de ASCII, evitando problemas com caracteres não imprimíveis.

7. Impacto Operacional para SOC

- **Estruturação de Dados:** A saída formatada permite a integração direta com ferramentas de análise de dados ou scripts em Python para automação.
- **Análise de Payload:** A capacidade de buscar strings dentro dos pacotes permite detetar ataques de injeção ou comandos C2 em protocolos de texto claro.

8. Problemas Encontrados e Resoluções

- **Ruído Visual:** A saída padrão do TShark é densa. **Resolução:** A combinação de `-Y` (filtros de exibição) com `-T fields` isola apenas os IoCs relevantes, como IPs de origem e métodos de requisição, aumentando a velocidade de análise.

9. Conclusão Técnica

O domínio dos operadores de comparação e da extração de campos eleva a capacidade do analista de SOC para realizar investigações minuciosas. Estas funcionalidades permitem tratar o tráfego de rede como uma base de dados consultável, essencial para a identificação de ameaças sofisticadas que operam na camada de aplicação.

Casos de Uso Forense e Manipulação de Dados com TShark

1. Contexto e Cenário

Após a análise estatística inicial, um analista de SOC deve realizar a extração de indicadores específicos para identificar ativos, vetores de ataque e ferramentas de exploração. Este relatório foca na extração de **Hostnames (DHCP)**, **Consultas DNS** e **User-Agents (HTTP)**, utilizando o TShark em conjunto com utilitários nativos do Linux para processamento de grandes volumes de dados.

2. Objetivo Técnico

Demonstrar a capacidade de isolar campos específicos de protocolos e aplicar lógica de pipeline (`awk`, `sort`, `uniq`) para gerar inteligência acionável, permitindo a identificação de ferramentas de varredura (Nmap, Wfuzz, Sqlmap) e nomes de host anómalos.

3. Implementação de Casos de Uso

3.1. Extração e Inventário de Hostnames (DHCP)

A extração de nomes de host através de pacotes DHCP permite o mapeamento de ativos que interagiram com a rede. A utilização do campo `dhcp.option.hostname` isola os nomes atribuídos pelas máquinas no processo de atribuição de IP.

Comando Estruturado:

```
Bash
tshark -r hostnames.pcapng -T fields -e dhcp.option.hostname | awk NF | sort -r | uniq -c | sort -r
```

3.2. Auditoria de Consultas DNS

A análise de nomes de domínio consultados é vital para detetar beacons de malware ou atividades de reconhecimento.

Análise de Resultados: Ao extrair `dns.qry.name`, o analista identifica domínios de infraestrutura legítima vs. subdomínios suspeitos. O uso de `sort -r` no final do pipeline garante que os domínios mais consultados apareçam no topo, evidenciando comunicações persistentes.

3.3. Identificação de Ferramentas via User-Agent (HTTP)

O campo `http.user_agent` é um dos principais indicadores para detetar ferramentas de automação e scanners de vulnerabilidades.

4. Resolução de Questões Técnicas

Abaixo, os resultados obtidos através da execução dos comandos de extração nos arquivos fornecidos:

Questão 1: Qual é o número total de nomes de host únicos?

```
ubuntu@...:~/Desktop/exercise-files$ tshark -r hostnames.pcapng -T fields -e dhcp.option.hostname | awk NF | sort -r | uniq -c | sort -r
 26 202-ac
 18 92-rkd
 14 93-sts-sec
 12 prus-pc
 10 90-tasd-sec
 10 60-alfb-sec2
  8 12-wew-sec
  6 temp_open
  6 off-admin-ass
  6 aminott
  6 60_old
  6 210-CAG
  4 T3400
  4 3-att-sec
```

Procedimento: Extração do campo DHCP seguido de contagem de linhas únicas.

Resposta: 30

Questão 2: Qual é o número total de ocorrências do nome de host "prus-pc"?

```
ubuntu@...:~/Desktop/exercise-files$ tshark -r hostnames.pcapng -T fields -e dhcp.option.hostname | awk NF | sort -r | uniq -c | sort -r | grep prus-pc
 12 prus-pc
ubuntu@...:~/Desktop/exercise-files$
```

Procedimento: Filtragem específica pelo nome de host alvo e contagem de frequência.

Resposta: 12

Questão 3: Qual é o número total de consultas para a consulta DNS mais comum?

[illegible]

Procedimento: Agrupamento de consultas DNS e identificação da maior contagem no topo da lista.

Resposta: 472

Questão 4: Qual é o número total de "agentes de usuário Wfuzz" detectados?

```

ubuntu@kali:~/Desktop/exercise-files$ tshark -r user-agents.pcap -T fields -e http.user_agent | awk NF | sort -r | uniq -c | sort -r
  9 Wfuzz/2.4
  6 Mozilla/5.0 (Windows; U; Windows NT 6.4; en-US) AppleWebKit/534.10 (KHTML, like Gecko) Chrome/8.0.552.237 Safari/534.10
  5 Mozilla/5.0 (X11; Ubuntu; Linux x86_64; rv:100.0) Gecko/20100101 Firefox/100.0
  5 Mozilla/5.0 (X11; Linux x86_64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/60.0.3112.32 Safari/537.36
  4 sqlmap/1.4#stable (http://sqlmap.org)
  3 Wfuzz/2.7
  3 Mozilla/5.0 (compatible; Nmap Scripting Engine; https://nmap.org/book/ns

```

Procedimento: Busca por padrões da ferramenta Wfuzz no campo User-Agent.

Resposta: 12

Questão 5: Qual é o "nome de host HTTP" das varreduras do nmap?

```
tshark -r user-agents.pcap -T fields -e ip.src -e ip.dst -e  
http.user.agent | awk NF | sort -r | uniq -c | sort -r
```

Procedimento: Correlação do tráfego identificado com o User-Agent do Nmap para extrair o Host Header correspondente.

Resposta: 172[.]16[.]172[.]129

5. Conclusão Técnica

A combinação do TShark com ferramentas de manipulação de texto do Linux (awk, sort, uniq) é uma competência crítica para o analista de SOC. Esta metodologia permite transformar milhares de pacotes brutos em listas de inventário e indicadores de ataque de forma rápida e precisa, facilitando a tomada de decisão durante a resposta a incidentes.